PaTH-VLA: Progress-and-Target Context for Long-Horizon Robot Policy Adaptation

Qinzhen Ma[1] Harry Yang[2] Changsheng Lu[2] Xiaogang Xu[3]
[1]Rice University [2]The Hong Kong University of Science and Technology [3]Zhejiang University

Abstract

Pretrained generalist robot policies often fail on multi-stage kitchen tasks in one of two ways: they lose track of where they are*, or act on the* wrong place*. Stage text and visual target prompts are popular remedies, but are evaluated as complete inference-time systems, obscuring what a policy learns from them. We run a controlled, pre-registered ablation on three RoboCasa365 policies, Xiaomi Robotics 1 (XR-1), Awomo-0.5I and Awomo-0.5V. Each is adapted with rank-8 LoRA for the same 1,000 updates under one of three auxiliary signals: progress (predicting the current stage, S), target (seeing the target box and predicting its location, BL), or both, the full PaTH-VLA recipe (F). Target boxes are pseudo-labelled automatically. Every policy is evaluated like the official checkpoint (original instruction, clean images) on 200 paired episodes over ten tasks, five unseen in adaptation. No pre-registered contrast is supported: given one signal, adding the other changes success by at most $+3.5$ pp (upper 95% bounds up to $+10.5$ pp), and for Awomo-0.5V adding box+location costs 8.5 pp (uncorrected $p = 0.03$; Holm-corrected $p = 0.06$). Against the official checkpoints, all XR-1 arms gain 6.5–7.5 pp, driven by the seen tasks, while the Awomo models lose up to 10–11 pp, most in the arms trained with drawn boxes; but the stage-only arm also loses on Awomo-0.5I's unseen tasks and no action-only arm was trained, so neither signal can be blamed. On training data, stage heads learn progress beyond a per-task prior and location heads little beyond a per-task constant; in a secondary test, feeding the predicted target back as a box costs XR-1 10 pp and helps neither Awomo model. We will release the labelling pipeline, the pre-registered protocol and all 2,400 logged episodes.*

1. Introduction

Generalist manipulation policies now ship with simulation benchmarks that score them on hundreds of multi-stage kitchen tasks [2, 20, 27]. On such tasks a policy can fail for two reasons that are easy to confuse: it may not know *where it is* in the task (it re-opens a drawer it has already opened, or heads for the stove before the pan is in hand), or it may not know *where to look* (it reaches for the wrong container, or releases the pan beside the burner). A growing line of work addresses both with extra context: stage or subtask text produced by a planner [1, 4, 23], and visual prompts such as marks, boxes and sketched paths drawn into the image [8, 13, 16, 26, 28]. These methods are typically evaluated as complete systems, which makes it hard to tell what the policy learned from the additional context and what is merely supplied to it at test time.

This paper asks a narrower question. Starting from three pretrained policies whose published success rates on ten RoboCasa365 tasks lie between 20% and 60%, we adapt each one for a fixed, small budget with lightweight adapters and one of three kinds of auxiliary context: *progress* supervision, i.e. predicting the current stage (S); *target* supervision, i.e. seeing the target box and predicting its location (BL); or both (F). We call the full recipe PaTH-VLA (Progress-and-Target Heads) and treat S and BL as its two single-signal ablations (Figure 1). Every adapted policy is evaluated like the official checkpoint (original instruction, clean images) on 200 paired episodes. Because the arms share the checkpoint, the data, the window draws, the optimizer and the update count, the two pre-registered contrasts F$-$BL and F$-$S measure what each kind of supervision contributes to ordinary-input behaviour (for XR-1, up to the initialisation of its new heads), not how well an inference-time prompting loop works.

Two design choices make the study practical. First, the supervision is cheap: stages come with the benchmark demonstrations, and target boxes are pseudo-labels from an open-vocabulary detector, a segmenter and a small vision-language model that picks the described instance (Figure 2); no human drew a box. Second, the protocol is paired and pre-registered: ten tasks, twenty seeds, four policies per model, the same seeds for all arms (byte-identical first frames on 191 of 200), a decision rule fixed before the first update, and every episode reported.

Our contributions are: (i) a controlled, cross-model ablation that separates stage supervision from target (box-and-location) supervision for pretrained generalist poli-

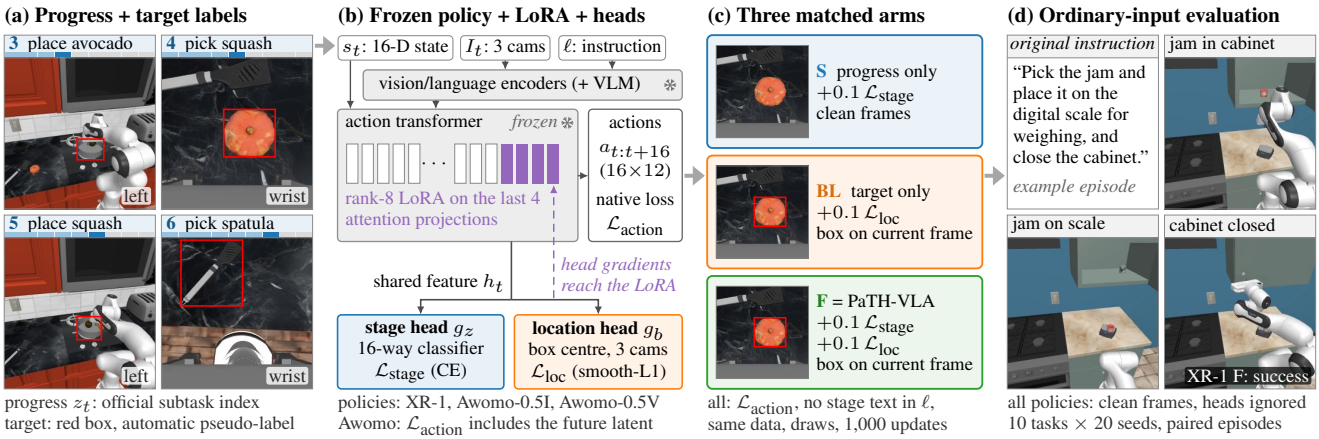


Figure 1. **Progress-and-target context for policy adaptation.** (a) A RoboCasa365 training demonstration (StirVegetables, stages 3–6 of 9) with the official subtask index z_t (progress) and an automatic pseudo-label box of the target (red); stage text is for the reader only. (b) Pretrained weights stay frozen; only rank-8 LoRA adapters and two linear heads on the shared feature h_t are trained. (c) The arms S, BL and F (PaTH-VLA) differ in the auxiliary signal; checkpoint, data, draws and updates are shared (XR-1's new heads were initialised differently; Section 6). (d) Evaluation with ordinary inputs: original instruction, clean frames, heads ignored (illustrative XR-1 F episode).

cies on composite RoboCasa365 tasks, including five tasks never seen in adaptation; (ii) an automatic target-labelling pipeline for subtask-annotated demonstrations that needs no human boxes and whose error profile we audit; and (iii) a pre-registered, paired evaluation protocol with 2,400 scored episodes whose outcome we state in Section 5. The outcome is a negative result: in no model did either signal measurably add to the other (point estimates within ± 4 pp except one *negative* one); against the official checkpoints the stage-only arm never differs significantly, and every nominally significant (uncorrected, secondary) loss is in an arm trained with boxes (pooled: up to -10 to -11 pp on the two world-action models), though without an action-only arm these losses cannot be attributed to the boxes. On training data the stage heads learn progress, the location heads little.

2. Related work

Generalist manipulation policies. After large-scale pretraining, vision-language-action models [5, 6, 12, 21, 23, 25] and Octo [22] map camera images, proprioception and a language instruction to action chunks. The three policies we adapt follow this recipe: XR-1 is a 5B vision-language-action model pretrained on more than 100K hours of real trajectories [27], and the two Awomo variants are world-action models that predict future image or video latents jointly with actions [2]. All three ship RoboCasa365 checkpoints [20], which lets us ask the same question across two model families with the official evaluation protocol. We adapt them with low-rank adapters [9] on a few late projections of the action transformer, so that all arms share one trainable set and one optimizer budget.

Visual prompts and intermediate representations. Marks drawn into an image are an effective interface for vision-language models (Set-of-Mark [28], PIVOT [19]) and for robot policies: MOKA grounds keypoints from marks [16], ReKep optimizes keypoint constraints [11], RT-Trajectory and HAMSTER condition on sketched 2D paths [8, 13], and RT-Affordance predicts affordance overlays as an intermediate representation [18]. VP-VLA is closest to our box channel: it overlays a crosshair and a box produced by an external planner and segmenter on an additional prompted image and trains the controller with a visual-grounding loss [26]. We keep the prompted-image idea but ask a narrower causal question: given a pretrained policy, does exposure to boxes during adaptation, with a location head, change ordinary-input performance?

Stage and language hierarchies. Decomposing a long-horizon instruction into stages is a recurring idea, whether the stages are produced by a language model (SayCan [1], Code as Policies [14], VoxPoser [10]) or predicted by the policy itself as language motions or semantic subtasks (RT-H [4], $\pi_{0.5}$ [23]). RoboCasa365 demonstrations carry per-frame subtask annotations [20], which we use as a stage-classification target rather than as an input. Auxiliary prediction targets are known to speed up policy learning in navigation [29]; whether they help an already pretrained manipulation policy on composite kitchen tasks, and which target matters, is what we measure.

3. Method

3.1. Setting

A pretrained policy π_θ receives, at control step t, three RGB cameras I_t (left and right agent views and a wrist view, 256×256), a proprioceptive state $s_t \in \mathbb{R}^{16}$ and the task instruction ℓ, and outputs a chunk of 16 actions $a_{t:t+16} \in \mathbb{R}^{16\times12}$ for a mobile manipulator. We never change this interface at test time. During adaptation we have N demonstrations of seen tasks, each frame annotated with the official subtask index $z_t \in \{0,\dots,K-1\}$ and, after our labelling step, a normalized box $b_t^{(c)} \in [0,1]^4$ of the current subtask's target in each camera c (or "not visible").

3.2. Adapters and heads

We insert rank-8 low-rank adapters [9] into the last four attention projections of the policy's action transformer (query/key/value and output projections for XR-1, the analogous projections of the Awomo diffusion transformers), and keep every pretrained weight frozen (Figure 1b). A forward hook captures the transformer's output feature $h_t \in \mathbb{R}^d$ (the first action token for XR-1, the mean over reference tokens for Awomo-0.5I, the mean over the last block's tokens for Awomo-0.5V). Two linear heads read h_t: a stage head $g_z(h_t) \in \mathbb{R}^{16}$ and a location head $g_b(h_t) \in [0,1]^{3\times2}$ that predicts the normalized centre of the target box in each camera. Both heads and the adapters are the only trainable parameters (1.2×10^5 for XR-1, 9.9×10^5 for Awomo-0.5I, 2.6×10^5 for Awomo-0.5V). Because the heads read the shared feature, their gradients flow into the adapters, i.e. auxiliary supervision can change the action pathway.

3.3. Losses and the three arms

All arms keep the native action objective of their policy: a masked flow-matching regression [15] of the action chunk for XR-1, and the same regression plus the future image or video latent regression for the Awomo models. On top of it we define

$$\mathcal{L}_{\text{stage}} = \mathrm{CE}\big(g_z(h_t), z_t\big), \tag{1}$$

$$\mathcal{L}_{\text{loc}} = \sum_{c\,:\,\text{visible}} \mathrm{SmoothL1}\big(g_b(h_t)^{(c)}, \mathrm{centre}(b_t^{(c)})\big), \tag{2}$$

and the total loss $\mathcal{L} = \mathcal{L}_{\text{action}} + 0.1\,\delta_S\,\mathcal{L}_{\text{stage}} + 0.1\,\delta_L\,\mathcal{L}_{\text{loc}}$, where $\delta_S, \delta_L \in \{0,1\}$ switch the two auxiliary terms. The box channel draws a 2-pixel red rectangle around $b_t^{(c)}$ on the *current* frame of every camera in which the target is visible; history frames and future-prediction targets stay clean. The three arms are

- **S** (progress only): $\delta_S{=}1$, no box, $\delta_L{=}0$;
- **BL** (target only): box drawn, $\delta_L{=}1$, $\delta_S{=}0$;
- **F** (full, PaTH-VLA): box drawn, $\delta_S{=}\delta_L{=}1$.

Within a model, S, BL and F start from the same checkpoint, see the same demonstration windows in the same order, and run the same number of optimizer updates with the same optimizer; only the switches above differ (and, for XR-1, the random initialisation of the new parameters; Section 6). No stage text is ever appended to the instruction, so the language input is identical across arms and identical to the official evaluation.

3.4. Target pseudo-labels

RoboCasa365 demonstrations contain subtask text and stage boundaries but no target boxes. We derive one target per stage description with a small rule table (the object to pick, the destination to place on or in, the part to actuate, the object to navigate to; 198 task-specific stage descriptions in our five training tasks, none left undefined) and label every training window automatically (Figure 2): Grounding DINO [17] proposes boxes for the target's class name, SAM 2.1 [24] refines each proposal into a tight mask box, and Qwen3-VL-4B [3] selects, per camera, the proposal that is the described *instance* ("the tupperware with only the fish in it", "the knob for the front-left burner") or declares the target not visible, using set-of-mark prompting [28] on an upscaled copy of the frame. Windows whose target is visible in no camera, and "task complete" stages, are dropped for all arms, so all arms train on the same windows. No human corrected any label; Section 4 audits them.

3.5. Ordinary-input evaluation

At test time the adapted policy is called exactly like the official checkpoint (original instruction, clean frames, native action chunking, official success function); the heads are computed but ignored, so we measure what the supervision taught the policy. Drawing the policy's own predicted target back into its frames is a separate, secondary experiment (Section 5).

4. Experimental protocol

The design below follows a protocol that was written down and hashed (SHA-256, to be released with the code) before the first optimizer update and the first scored episode, and stored with the run logs rather than deposited with a registry (text in the supplement). Deviations and post-hoc analyses are listed at the end of this section.

Models and compute. XR-1 [27], Awomo-0.5I and Awomo-0.5V [2] (three models from two families), each with its official RoboCasa365 checkpoint and native inference code wrapped in LeRobot [7]. All runs use NVIDIA H100 PCIe GPUs; a training run takes 40–41 minutes for XR-1 and 106–173 minutes for the Awomo models, and an evaluation episode 0.5–14 minutes (policy means 3.3–6.6; 207 hours summed over all 2,400 episodes).

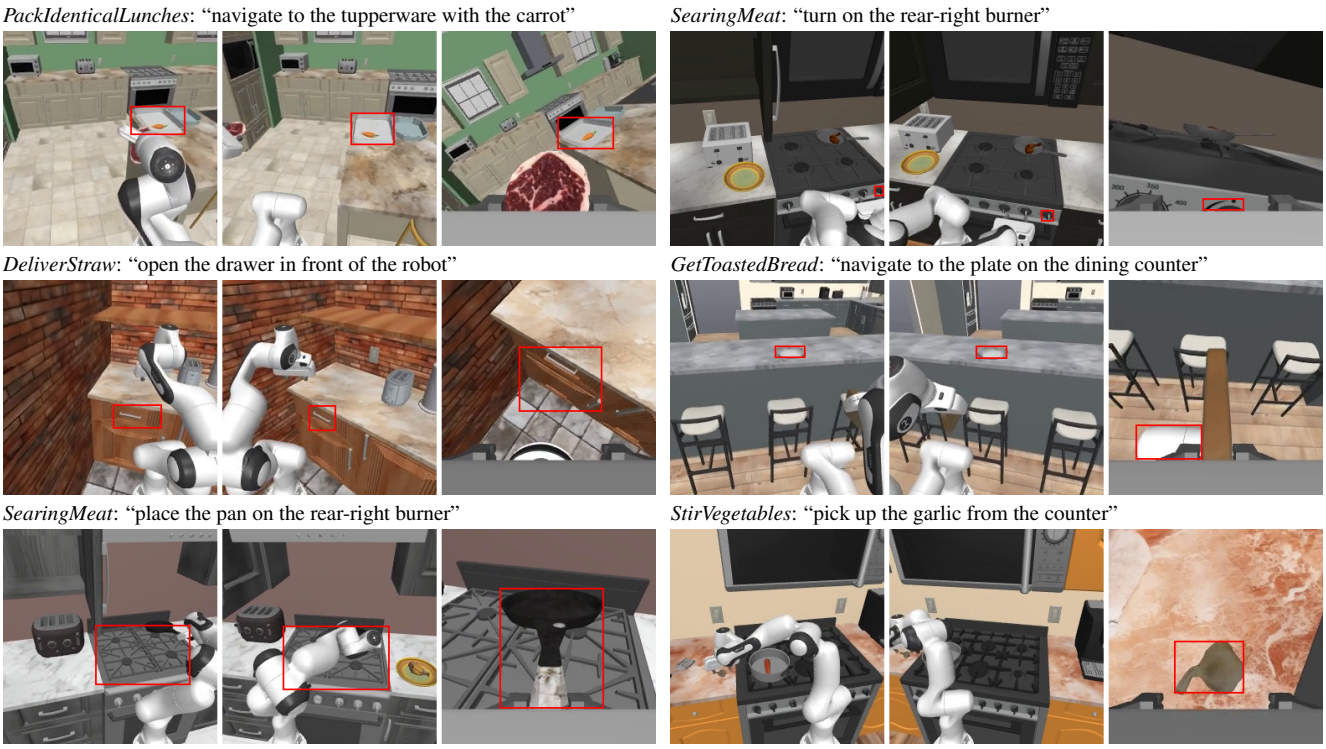


Figure 2. **Automatic target pseudo-labels.** Six training windows (task and stage description above the left, right and wrist cameras). The red box is the Grounding DINO → SAM 2.1 proposal that Qwen3-VL selected as the described instance; its centre is the location target. The pipeline resolves instance references (the container *with the carrot*, the knob of the *rear-right* burner) without human input, and also makes typical mistakes: for "place the pan on the rear-right burner" both agent views box the whole cooktop and the wrist view the held pan, and in the plate example the wrist view boxes a chair. No label was corrected by hand.

Tasks. Ten RoboCasa365 composite tasks, five from the composite-seen split (DeliverStraw, GetToastedBread, PackIdenticalLunches, SearingMeat, StirVegetables) and five from the composite-unseen split (ArrangeTea, CuttingToolSelection, RecycleBottlesByType, WaffleReheat, WeighIngredients). The ten were chosen before any run from the published per-task success rates, requiring every model to be between 20% and 60% on every task, so that both gains and losses are measurable; this is a performance-screened, not random, subset. Exactly five seen tasks pass this screen; of the six unseen tasks that pass, PortionHotDogs (exactly 20% for all three models) was left out. The five unseen tasks are never used for training, labelling or tuning.

Training data and budget. Per seen task we fix 20 training and 5 development demonstrations by SHA-256 ranking of the official human demonstrations (100 + 25 episodes). Every anchor frame that is a multiple of 16 with at least 32 future frames forms a window (8,931 training windows); after labelling, 8,174 windows with a visible target remain and are shared by all arms. Each run performs 1,000 optimizer updates of AdamW (learning rate 10^{-4}, weight decay 0.01, one window per micro-batch, eight micro-batches per update, seed 20261106), i.e. 8,000 task-balanced draws with replacement (1,600 per task) that reach 4,977 distinct windows, 60.9% of the panel (DeliverStraw 77%, GetToastedBread 62%, PackIdenticalLunches 58%, SearingMeat 50%, StirVegetables 69%). The update count was fixed from a timing calibration before launch and is identical for S, BL and F of every model.

Evaluation. Every policy (official checkpoint, S, BL, F; twelve in total) is evaluated on the same 200 episodes: ten tasks × twenty fixed seeds (0–19). A seed is meant to fix the kitchen layout, style, object instances and initial state, so the comparison is paired by (task, seed). Hashing the first observation shows that it is byte-identical across all twelve policies for 191 of the 200 seeds; for the other 9 it came in two variants across policies, and for two of them the initial robot base pose also differed (seeds listed in the supplement). We keep all 200 pre-registered pairs and, post hoc, repeat every contrast on the 191 identical seeds. We use the official per-task horizon (1,200 to 4,350 steps), the official success function, native 16-step action chunks and a fixed policy RNG seed (4242 plus the replanning index).

All 2,400 episodes are reported; no task, seed or checkpoint was replaced after seeing results, and only the final checkpoint of each run is evaluated.

Statistics. The two primary contrasts per model are F−BL (the contribution of stage supervision given target supervision) and F−S (the contribution of target supervision given stage supervision). For each we report the paired success difference in percentage points pooled over the 200 pairs, a 95% interval from a task-stratified paired bootstrap (10,000 draws), an exact two-sided McNemar test on the discordant pairs, and Holm correction over the two primary contrasts of a model: the contrast with the smaller p is judged on its 97.5% interval and doubled p, the other, only if the first is rejected, on its 95% interval. We pre-specified that a contrast counts as supported only if its corrected interval excludes zero and the point estimate is at least $+5$ pp. Differences to the official checkpoint are secondary and uncorrected, because by design no matched action-only arm was trained (Section 6).

Label audit. A simple random sample of 48 labelled windows (144 camera views) was inspected visually after labelling by the AI agent that executed the study (see the LLM Usage Statement). Of the 103 camera views with a visible-target label, 93 (90%) bound the described target (eight of them the whole appliance rather than the named part); the ten errors are six robot links, three wrong instances among duplicates and one unrelated object (details in the supplement).

Deviations and post-hoc analyses. Seeds did not always fix the scene (above), XR-1's new heads did not start identically in the three arms (Section 6), and the optional feedback study fed back the box only, not the stage. 18 Awomo-0.5V official-checkpoint episodes that crashed (GPU out of memory) were rerun from scratch, a decision taken after partial results had been seen, instead of being reported as errors; 5 reruns succeeded, so counting the crashes as failures gives 83 instead of 88 successes and changes only the contrasts against that checkpoint (Section 5). Nine duplicate official-checkpoint episodes from a smoke test and an aborted first launch are not among the 2,400; each matches its released counterpart in outcome, length and first frame. Not pre-registered: the scene-identity and crash sensitivity analyses, the S−BL contrast, the per-task significance tests and the head diagnostics.

5. Results

Primary contrasts: no evidence that the signals add up. Table 1 and Figure 3b show the pre-registered contrasts. Adding stage supervision on top of target supervision (F−BL) changes success by $+1.0$ pp for XR-1, $+3.5$ pp for Awomo-0.5I and -3.5 pp for Awomo-0.5V; every interval covers zero and the discordant pairs split almost evenly. Adding target supervision on top of stage supervision (F−S) is $+1.0$ pp for XR-1 and -1.5 pp for Awomo-0.5I, again indistinguishable from zero, and -8.5 pp for Awomo-0.5V (95% CI $[-15.5, -1.5]$, 19 wins against 36 losses, exact $p = 0.030$). Under the pre-registered rule no contrast is supported, and the upper 95% bounds ($+6.5$ to $+10.5$ pp in four of the six contrasts) do not exclude gains of the pre-registered $+5$ pp size. For the video world-action model, adding box+location lowered success: its Holm-corrected interval $[-16.5, -1.0]$ excludes zero, but the Holm-corrected McNemar test does not ($p = 0.060$), and each arm has one training seed. On the 191 identical seeds (supplement) it is -9.4 pp ($p = 0.015$, Holm $p = 0.031$); every other primary interval still covers zero and no contrast becomes supported.

Model	Contrast	Δ (pp)	95% CI	W/L	p
XR-1	**F−BL**	$+1.0$	$[-6.0, +8.0]$	28/26	0.892
	F−S	$+1.0$	$[-5.5, +7.5]$	25/23	0.885
	F−official	$+7.5$	$[+0.5, +14.5]$	33/18	0.049
	S−official	$+6.5$	$[-0.5, +13.5]$	33/20	0.098
	BL−official	$+6.5$	$[0.0, +13.0]$	31/18	0.085
Awomo-0.5I	**F−BL**	$+3.5$	$[-3.5, +10.5]$	32/25	0.427
	F−S	-1.5	$[-9.5, +6.5]$	33/36	0.810
	F−official	-6.5	$[-14.0, +1.0]$	24/37	0.124
	S−official	-5.0	$[-13.0, +3.0]$	33/43	0.302
	BL−official	-10.0	$[-18.5, -1.5]$	30/50	0.033
Awomo-0.5V	**F−BL**	-3.5	$[-11.0, +4.0]$	29/36	0.457
	F−S	-8.5	$[-15.5, -1.5]$	19/36	0.030
	F−official	-11.0	$[-19.5, -3.0]$	27/49	0.015
	S−official	-2.5	$[-10.5, +5.5]$	31/36	0.625
	BL−official	-7.5	$[-15.5, 0.0]$	26/41	0.086

Table 1. **Paired contrasts.** Pooled over 200 (task, seed) pairs per model: success-rate difference, task-stratified paired-bootstrap 95% interval, wins/losses among discordant pairs, exact McNemar p (uncorrected). Bold rows are the two pre-registered primary contrasts; their Holm-corrected verdicts are in the text.

Against the official checkpoints: one model gains, two lose. For XR-1 all three arms are above the official checkpoint by a similar margin: $+6.5$ pp (S), $+6.5$ pp (BL) and $+7.5$ pp (F; CI $[+0.5, +14.5]$, 33 rescues against 18 regressions, $p = 0.049$). The gain comes from the seen tasks ($+13$ pp for F, CI $[+2, +24]$) and essentially vanishes on the five unseen tasks ($+2$ pp, CI $[-6, +10]$; split contrasts in the supplement). Because the three arms gain alike although their auxiliary signals differ, and because the gain tracks the tasks whose demonstrations were used, the most parsimonious reading is that XR-1 profits from the extra action regression on seen-task data rather than from either

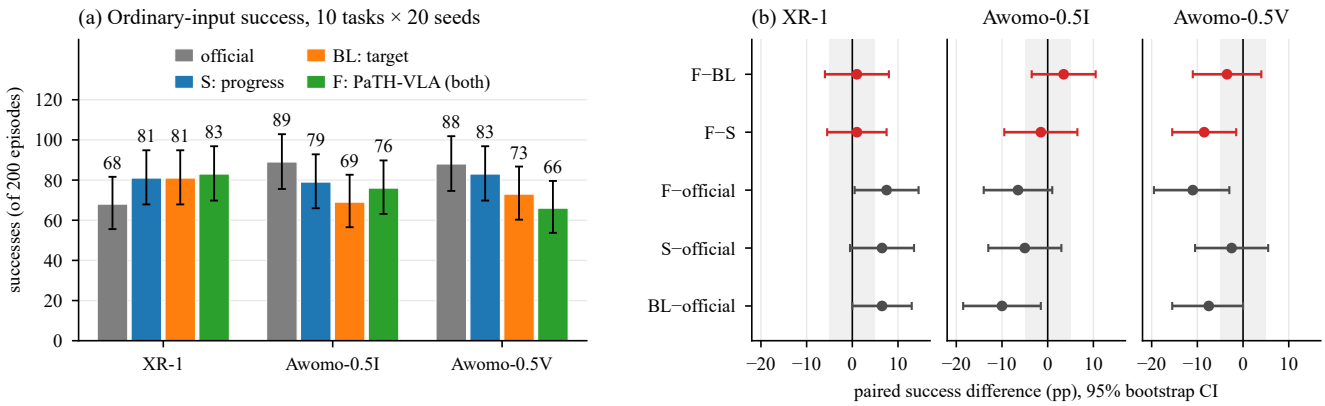


Figure 3. **Main result.** (a) Ordinary-input successes of the official checkpoint and the three adapted arms of each model on the same 200 seeded episodes (10 tasks × 20 seeds), with Wilson 95% intervals. (b) Paired success differences with task-stratified paired-bootstrap 95% intervals (uncorrected); red: the pre-registered primary contrasts; grey band: ±5 pp. No primary contrast is positive and significant; the only primary contrast whose Holm-corrected interval (text) excludes zero, F−S for Awomo-0.5V, is negative.

auxiliary signal, or partly regresses to the mean from an official run that was weakest on exactly these tasks (2 of 20 on GetToastedBread, against ≥20% published); without an action-only arm we cannot separate these readings. For the Awomo models the same adaptation is neutral at best. Awomo-0.5I drops by 5.0 (S), 10.0 (BL; CI $[-18.5, -1.5]$, $p = 0.033$, or 0.054 on the 191 identical seeds) and 6.5 pp (F); the losses concentrate on the unseen tasks (S: −11 pp, CI $[-22, 0]$, $p = 0.090$; BL: −18 pp, CI $[-29, -7]$, $p = 0.004$; F: −13 pp, $p = 0.029$) while the seen tasks are flat. Awomo-0.5V drops by 2.5 (S), 7.5 (BL) and 11.0 pp (F; CI $[-19.5, -3.0]$, $p = 0.015$), on seen as well as unseen tasks; with its 18 crashed episodes counted as failures (Section 4) these become 0.0, −5.0 and −8.5 pp (F: $p = 0.068$).

Per-task view: broad shifts, noisy cells. Figure 4 splits these differences by task. For XR-1, all three arms gain 2–4 successes on each of the four seen tasks where the official checkpoint is weakest (2–6 of 20), change by −3, −3 and 0 on DeliverStraw, its strongest seen task (10/20), and stay within $[-2, +4]$ on unseen tasks. Awomo-0.5I's unseen losses are spread, not concentrated: 14 of its 15 unseen cells are negative, and F loses 2–3 successes on every unseen task. Awomo-0.5V F loses on seven of ten tasks; without its largest loss (PackIdenticalLunches, −6) the other nine still sum to −16. Single cells are noisy: on 20 pairs $p < 0.05$ needs a difference of at least 6, and at 6 no opposite discordant pair; each of the four cells at −6 has one ($p = 0.07$), so none reaches $p < 0.05$. The three arms also share one official baseline, so an unlucky official run on a task would appear as a gain in every arm; their agreement is not independent evidence.

		XR-1			Awomo-0.5I			Awomo-0.5V		
		S	BL	F	S	BL	F	S	BL	F
seen	Straw	−3	−3	0	+1	+2	−2	−4	−5	−3
	Toast	+3	+4	+4	0	−2	−2	−3	−5	−4
	Lunch	+3	+3	+3	0	+4	+4	+3	+1	−6
	Sear	+4	+2	+4	+4	0	+2	−2	−2	−2
	Stir	+2	+3	+2	−4	−6	−2	+4	+1	+1
	Σ	**+9**	**+9**	**+13**	**+1**	**−2**	**0**	**−2**	**−10**	**−14**
unseen	Tea	+4	−1	−1	+1	−2	−3	−2	−1	−5
	Cut	+3	+4	+2	−3	−1	−3	−1	−4	−2
	Recycle	−1	+2	+1	−4	−3	−2	−1	−1	−5
	Waffle	0	−2	0	−2	−6	−3	+2	+1	+2
	Weigh	−2	+1	0	−3	−6	−2	−1	0	+2
	Σ	**+4**	**+4**	**+2**	**−11**	**−18**	**−13**	**−3**	**−5**	**−8**

Figure 4. **Per-task differences to the official checkpoint.** Each cell: successes of an adapted arm minus those of the official checkpoint on the same 20 seeded episodes of one task (one success = 5 pp; blue: gain, red: loss). Σ: sum over the five tasks of a block (100 paired episodes; one success = 1 pp). Only seen tasks supplied training demonstrations. No cell reaches $p < 0.05$ (exact McNemar, uncorrected); counts are in the supplement.

Where the losses fall. The stage-only arm is never significantly below the official checkpoint (+6.5, −5.0, −2.5 pp), and every nominally significant loss is in an arm that drew boxes during training (pooled: BL of Awomo-0.5I and F of Awomo-0.5V, the latter only with the reruns; on the unseen tasks also F of Awomo-0.5I). These are uncorrected comparisons with a shared baseline, and they do not isolate the box: the stage-only intervals reach down to −13.0 (Awomo-0.5I) and −10.5 pp (Awomo-0.5V), and the direct contrasts separate the arms only for Awomo-0.5V (F−S, above); for Awomo-0.5I, F−S is −1.5 pp and S−BL +5.0 pp ($p = 0.22$). One untested explanation: both

	XR-1	Awomo-0.5I	Awomo-0.5V
Stage cross-entropy (nats), S / F			
First 50 updates‡	3.20 / 2.66	2.63 / 2.63	2.50 / 2.49
Last 100 updates	1.45 / 1.38	1.29 / 1.31	1.69 / 1.69
Uniform: 16 / task's stages	*all models:* 2.77 / 1.94		
Prior: task / task + time	*all models:* 1.87 / 1.10†		
First visits vs. task prior	−0.33 / −0.40	−0.45 / −0.44	−0.08 / −0.09
Location smooth-L1 ($\times 10^{-3}$), BL / F			
First 50 updates‡	22.8 / 31.7	18.2 / 18.3	18.7 / 18.7
Last 100 updates	15.2 / 14.8	17.2 / 17.4	15.8 / 15.6
Mean centre: global / task	*all models:* 17.4 / 16.4		
Mean centre: task + stage	*all models:* 13.9 / 14.9†		
First visits vs. task centre	−0.4 / −0.9	+0.7 / +1.3	−0.1 / −0.2
Last 100 updates, change vs. S (%), BL / F			
Action loss	+1.7 / +2.9	−0.8 / +1.7	+0.1 / +1.1
Future-latent loss	–	+2.3 / +4.0	+0.5 / +2.1

Table 2. **Do the auxiliary heads learn?** Means of the logged training losses (1,000 updates). References are fitted on the training panel and scored on the same draws. First visits: head minus reference on the 296 late draws (37%) whose window had not been drawn before. Bottom: paired change vs. S. Paired standard errors: stage ≤ 0.05, location ≤ 0.7, action $\leq 0.9\%$, future latent $\leq 0.2\%$. †Fitted on held-out demonstrations; elapsed time in 128-frame bins. ‡XR-1's heads were initialised differently in each arm.

Awomo models regress a *clean* future image or video latent from a *painted* current frame, so the box is a train-time-only input to their world-model pathway, whereas XR-1's frozen vision-language backbone feeds only its action transformer. It does not account for Awomo-0.5I's stage-only unseen loss, and generic forgetting from seen-task adaptation, or interference from the location loss (Table 2), fit as well.

Do the auxiliary heads learn? In the training logs (Table 2) the stage heads learn: their cross-entropy falls from 2.49–3.20 nats (first 50 updates) to 1.29–1.69 nats (last 100), below the 1.87 nats of the best predictor that knows only the task, though above the 1.10 of one that also knows the elapsed time. The location heads learn little: XR-1's and Awomo-0.5V's beat a per-task constant centre by at most 1.6×10^{-3}, Awomo-0.5I's stays near a global constant, and none beats the in-sample constant per task and stage (the best, XR-1's F head, ties the held-out one). These are training-set losses, not held-out accuracy: only 37% of the late windows were first visits, and on those the stage heads stay below the task prior (Awomo-0.5V by just 0.08–0.09 nats) while no location head reliably beats the per-task constant. Nor do they show that a learned head helps the policy: the heads are ignored at test time, and adding either signal to the other slightly raised the training action loss in every model (F vs. BL $+1.0$ to $+2.6\%$, F vs. S $+1.1$ to $+2.9\%$).

Model	n	Off.	F	FB	Δ	95% CI	p
XR-1	200	68	83	63	-10.0	$[-17.5, -2.5]$	0.013
Awomo-0.5I	200	89	76	66	-5.0	$[-12.5, +2.5]$	0.260
Awomo-0.5V	200	88	66	58	-4.0	$[-11.5, +3.5]$	0.382

Table 3. **Feeding the policy's own target back (secondary).** Successes of the official checkpoint (Off.), of the PaTH-VLA checkpoint with ordinary inputs (F) and of the same checkpoint with its location-head prediction from the previous query drawn as a fixed 32×32-pixel box on the current frame of every camera (FB; none at the first query, instruction unchanged), on the same n seeded episodes. Δ: FB−F success difference (pp) with task-stratified paired-bootstrap 95% interval and exact McNemar p (uncorrected). Within an episode the predicted centre has a median RMS spread of 32 px (XR-1), 11 px (Awomo-0.5I) and 12 px (Awomo-0.5V), against 59 px for the labelled target within a training demonstration.

How episodes fail. Every failure in all 2,400 episodes is a horizon timeout; no policy terminated early or triggered an infeasible state. Successful XR-1 episodes are shorter for F and BL than for the official checkpoint (median 1,281 and 1,332 vs. 1,365 steps) but longer for S (1,427); all Awomo-0.5I arms are slower (1,351–1,585 vs. 1,314), and for Awomo-0.5V the direction differs by arm (per-policy statistics in the supplement). Figure 5 shows two selected paired episodes: an XR-1 rescue on an unseen task, and an Awomo-0.5V F regression on PackIdenticalLunches.

Secondary: feeding the policy's own target back. In a separate evaluation (FB, Table 3), each PaTH-VLA checkpoint replays the same 200 episodes while its location head's prediction from the previous query is drawn as a fixed 32×32-pixel box on every camera's current frame (the stage head is not used). Relative to ordinary inputs, FB changes success by -10.0 pp for XR-1 (95% CI $[-17.5, -2.5]$, $p = 0.013$; -10.2 pp on the 196 pairs whose first frames match), and by -5.0 pp (CI $[-12.5, +2.5]$, $p = 0.260$) and -4.0 pp (CI $[-11.5, +3.5]$, $p = 0.382$) for Awomo-0.5I and Awomo-0.5V. For XR-1 this erases F's gain over the official checkpoint (FB−official: -2.5 pp, CI $[-9.0, +4.0]$). Training instead drew tight label boxes only where the target was visible, so FB tests the head's output as a prompt, not the value of accurate boxes. An earlier Awomo-0.5V run that read the head from the wrong feature was discarded unanalysed (73 episodes).

6. Limitations

Scope of the causal claim. The contrasts F−BL and F−S measure the value of one kind of supervision *given* the other, under one adaptation budget (1,000 updates, rank-8 adapters on four projections, one training seed). They do not establish that the effects are additive, and a different

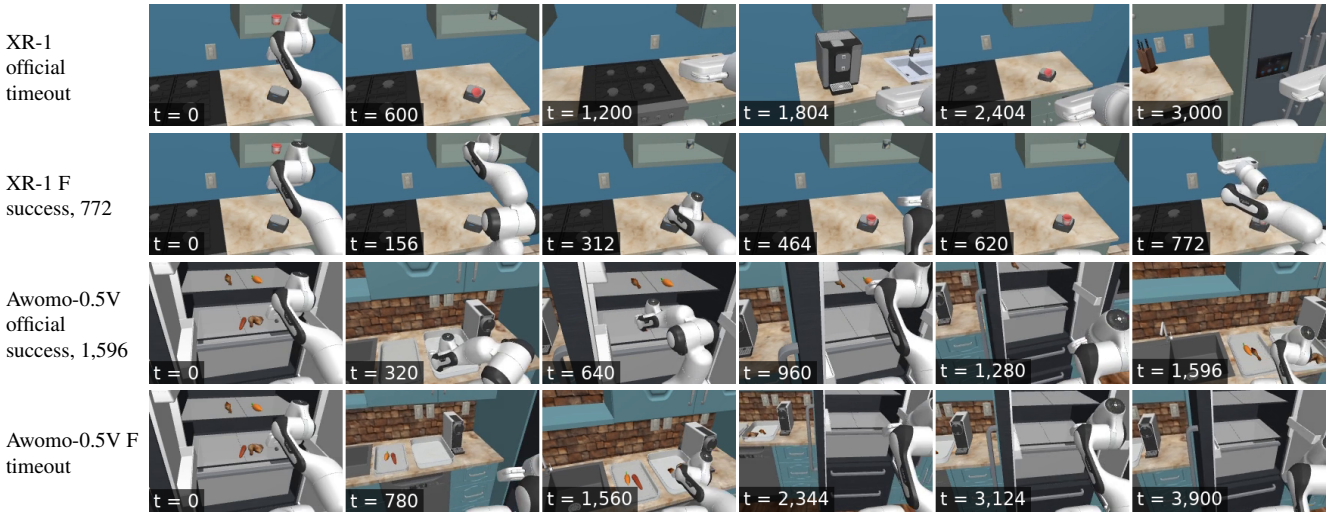


Figure 5. **Selected qualitative rollouts.** Agent view; step index in each frame. Rows 1–2: WeighIngredients, seed 12 (unseen; F and the official XR-1 tie at 8/20 on this task). The official policy wanders until the horizon; F completes the task (jam on the scale, cabinet closed) at step 772. Rows 3–4: PackIdenticalLunches, seed 17. The official Awomo-0.5V succeeds at step 1,596; its F version carries food to the counter, then drifts to the wrong drawers and times out, whereas BL succeeds (step 1,673). Each pair starts from identical scenes.

budget, adapter placement, loss weight or training seed could move them. We report one training seed per arm because the compute window did not allow three; the twenty evaluation seeds per task bound the evaluation noise but not the training noise. For XR-1, moreover, the new heads did not start from the same random draw in the three arms: at update 0, BL and F (identical inputs) have equal action losses but location losses up to 0.065 apart, and S and F stage losses up to 3.9 nats apart (Awomo: 0 and $\leq$0.03); its LoRA A matrices may differ by the same seeding path. The action pathway still starts identical because LoRA B starts at zero, but XR-1's between-arm differences also contain seed-to-seed variation of the new parameters.

No matched action-only arm. We compare adapted policies with the official checkpoint, not with a policy adapted on the same windows without auxiliary losses. Differences to it therefore include the effect of extra action-regression training on seen-task demonstrations and must not be read as the benefit of the auxiliary supervision alone; only the within-model contrasts isolate it.

Pseudo-labels. Target boxes come from an automatic pipeline whose audited error rate is reported in Section 4: it sometimes boxes a robot link, the wrong instance, an unrelated object, or the whole appliance instead of the named part, and the location target is a 2D box centre, not a contact point. Our conclusions about the box channel are conclusions about *this* label quality.

Benchmark and task selection. The ten tasks were screened by published success rate; results on them do not transfer to the whole benchmark, and the composite-unseen tasks test transfer of what was learned on five seen tasks, not open-world generalization. Simulation only; no real-robot evaluation.

7. Conclusion

We asked what three pretrained RoboCasa365 policies learn from progress and target supervision under a matched adaptation budget, evaluating them with ordinary inputs on the same 200 seeded episodes. We found no evidence that the two signals are complementary, a small gain for XR-1 (F: $+7.5$ pp, uncorrected $p = 0.049$) that we cannot attribute to either signal, and losses for the world-action models that are largest in, but not confined to, the arms trained with drawn boxes. Within our budget a stage head never measurably hurt and learned progress, whereas adding target supervision to it never helped and, for Awomo-0.5V, lowered success. Whether visual prompts present only at training time are a liability for policies that predict future frames is a hypothesis our design cannot test. The labelling pipeline, the pre-registered protocol and every logged episode will be released so that the same question can be asked of the next generation of policies with a larger adaptation budget, more training seeds and a matched action-only arm.

LLM Usage Statement

LLM agents worked under the authors' direction. OpenAI Codex prepared the initial study plan, the model wrappers

and the training framework. Claude Fable 5.1 (Anthropic) executed the study: it drafted the pre-registration, wrote the labelling, evaluation and primary analysis code, adapted the training code, ran all experiments, performed the visual label audit (Section 4), made the figures and tables, compiled references and drafted the text. Claude Opus 5.5 (Anthropic) redrew Figure 1, added the per-task, head, sensitivity and feedback analyses, and revised and checked the text. Qwen3-VL-4B-Instruct is part of the labelling pipeline (Section 3). All experimental decisions, the protocol and the claims are the authors' responsibility. Every number is computed by analysis scripts (to be released) from the episode, training and label logs, except the label-audit counts and three counts from cluster run logs (see the supplement).

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, pages 287–318, 2022. 1, 2

[2] Awomo-WAM Team. Awomo-0.5: World action model pretrained at scale. Hugging Face organisation `Auwomo`, 2026. Image (Awomo-0.5I) and video (Awomo-0.5V) variants; RoboCasa365 checkpoints `Auwomo/Awomo-0.5{I,V}-RoboCasa365`; technical report not yet released at the time of writing. 1, 2, 3

[3] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. Qwen Team, Alibaba Group. 3

[4] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: Action hierarchies using language. In *Robotics: Science and Systems (RSS)*, 2024. 1, 2

[5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025. 2

[6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, pages 2165–2183, 2023. 2

[7] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. GitHub repository `huggingface/lerobot`, 2024. 3

[8] Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, Priya Sundaresan, Peng Xu, Hao Su, Karol Hausman, Chelsea Finn, Quan Vuong, and Ted Xiao. RT-Trajectory: Robotic task generalization via hindsight trajectory sketches. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 2, 3

[10] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, pages 540–562, 2023. 2

[11] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *Conference on Robot Learning (CoRL)*, pages 4573–4602, 2024. 2

[12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, pages 2679–2713, 2024. 2

[13] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. HAMSTER: Hierarchical action models for open-world robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. 1, 2

[14] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 2

[15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 3

[16] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Robotics: Science and Systems (RSS)*, 2024. 1, 2

[17] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision (ECCV)*, 2024. 3

[18] Soroush Nasiriany, Sean Kirmani, Tianli Ding, Laura Smith, Yuke Zhu, Danny Driess, Dorsa Sadigh, and Ted Xiao. RT-Affordance: Affordances are versatile intermediate representations for robot manipulation. *arXiv preprint arXiv:2411.02704*, 2024. 2

[19] Soroush Nasiriany, Fei Xia, Wenhao Yu, Ted Xiao, Jacky Liang, Ishita Dasgupta, Annie Xie, Danny Driess, Ayzaan Wahid, Zhuo Xu, Quan Vuong, Tingnan Zhang, Tsang-Wei Edward Lee, Kuang-Huei Lee, Peng Xu, Sean Kirmani, Yuke Zhu, Andy Zeng, Karol Hausman, Nicolas Heess, Chelsea Finn, Sergey Levine, and Brian Ichter. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs. In *International Conference on Machine Learning (ICML)*, pages 37321–37341, 2024. 2

[20] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026. 1, 2

[21] NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 2

[22] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. 2

[23] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2

[24] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations (ICLR)*, 2025. 3

[25] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[26] Zixuan Wang, Yuxin Chen, Yuqi Liu, Jinhui Ye, Pengguang Chen, Changsheng Lu, Shu Liu, Bei Yu, and Jiaya Jia. VP-VLA: Visual prompting as an interface for vision-language-action models. *arXiv preprint arXiv:2603.22003*, 2026. 1, 2

[27] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026. Model checkpoints: Hugging Face `XiaomiRobotics/Xiaomi-Robotics-1-5B`; referred to as XR-1 on the model card. 1, 2, 3

[28] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023. 1, 2, 3

[29] Joel Ye, Dhruv Batra, Erik Wijmans, and Abhishek Das. Auxiliary tasks speed up learning PointGoal navigation. In *Conference on Robot Learning (CoRL)*, pages 498–516, 2020. 2